

An Automated PHP Legacy Code Migration Pipeline with Integrated ISO/IEC 27001:2022 Security Compliance Analysis

1st Muhammad Farrel Akbar
*Department of Electrical and
Information Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
muhammadfarrelakbar@mail.ugm.ac.id*

2nd Dani Adhipta
*Department of Electrical and
Information Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
dani@ugm.ac.id*

3rd Ir. Addin Suwastono
*Department of Electrical and
Information Engineering
Universitas Gadjah Mada
Yogyakarta, Indonesia
adyn@ugm.ac.id*

Abstract—Most websites still run PHP applications on version 7.x or older that no longer receive official security updates, leaving systems exposed to an ever-growing accumulation of known vulnerabilities. Manual migration across hundreds of source files is impractical, and existing transformation tools do not incorporate security compliance checks as an integrated part of the migration workflow. Organizations operating under ISO/IEC 27001:2022 must also perform security audits as a separate post-migration step, adding to the operational burden. This research builds a Python-based pipeline that integrates automatic conversion of legacy PHP web applications to a configurable target PHP version with ISO/IEC 27001:2022 secure coding compliance checks in a single, fully local, open-source workflow. The pipeline combines Rector for AST-based code transformation, Semgrep for security scanning, PHPStan for post-conversion type validation, and five open-source large language models running locally via Ollama for semantic vulnerability analysis and remediation suggestions. Case studies were conducted on PHP codebases from FT UGM (245 files) and CBT DTI UGM (326 files). Rector achieved conversion rates of 97.6% and 91.7% respectively, with 100% post-conversion syntax validity in both cases. A security finding delta of $\Delta F = 0$ confirmed that the conversion process introduced no new vulnerabilities. Automatically generated JSON reports include per-finding mappings to ISO/IEC 27001:2022 Annex A controls and serve directly as audit documentation. Among five evaluated LLMs, Qwen2.5-Coder 7b is recommended for pipeline integration, consistently achieving a Syntax Validity Rate of 100% across all experimental conditions.

Index Terms—PHP migration, secure coding, ISO/IEC 27001:2022, large language model, AST code transformation

I. INTRODUCTION

PHP powers 71.7% of all websites with a known server-side language [1], yet nearly half of those sites run versions that no longer receive official security updates: 33.1% on PHP 7.x and 8.7% on PHP 5.x [1]. Official support for PHP 7.4 ended on 28 November 2022 [2], leaving unpatched systems permanently exposed to vulnerabilities catalogued by NIST NVD [3]. Migrating to PHP 8.x is not a simple search-and-replace operation: breaking changes—including the removal of the entire `ext/mysql` extension, a stricter type system, and altered error handling—affect large codebases in ways that require structural analysis, not merely textual substitution [2].

Available tools address parts of the problem in isolation. Rector [4] handles AST-based syntactic transformations reliably, as confirmed at production scale by Chen et al. [5], but cannot replace `mysql_*()` calls that require semantic understanding of database access logic. Semgrep [6] and PHPStan [7] detect security patterns and type errors, but neither integrates into a migration workflow. Crucially, none of these tools produces ISO/IEC 27001:2022 compliance evidence as a by-product of migration.

Organizations pursuing ISO/IEC 27001:2022 certification [8] must document adherence to Annex A.8.25 (secure development lifecycle), A.8.28 (secure coding), and A.8.29 (security testing in development), requiring audit evidence that code migration was conducted in accordance with those controls—a burden currently met through manual work entirely separate from the migration process.

Advances in locally deployable large language models [9] now make it feasible to provide semantic analysis without transmitting source code to external services. Li et al. [10] showed that combining LLMs with static analysis detects more vulnerabilities than either approach alone. However, existing LLM-assisted approaches [11] [12] use cloud-based models and do not address version migration or compliance standards.

This paper makes the following contributions: (1) a Python-based pipeline that combines Rector, Semgrep, PHPStan, and five locally running open-source LLMs into a single automated workflow for PHP legacy code migration with integrated ISO/IEC 27001:2022 compliance analysis; (2) empirical validation on two real-world PHP codebases totaling 571 files; and (3) a systematic comparison of five open-source LLMs on PHP security remediation tasks, establishing Syntax Validity Rate (SVR) as the most reliable evaluation metric for this context.

II. RELATED WORK

Automated PHP security remediation. Merlo et al. [13] demonstrated in 2007 that automated static-and-dynamic analysis can protect legacy PHP against SQL injection without altering functionality, achieving zero false positives on phpBB

2.0.0. Their work confirms that automated PHP security analysis is feasible; the gap is that no comprehensive solution operating against modern security standards has emerged in the intervening decades.

LLM-based vulnerability detection. Akuthota et al. [11] showed that GPT-3.5-Turbo can identify common vulnerability patterns via prompt engineering alone, with accuracy degrading as pattern complexity increases. Li et al. [14] improved results by combining patch history with structured static analysis. Both studies use cloud APIs, creating a data privacy barrier for institutions managing confidential source code. Li, Dutta, and Naik [10] introduced IRIS, which combines LLMs with CodeQL for whole-repository taint analysis, detecting 55 of 120 CWE-Bench-Java vulnerabilities versus 27 by CodeQL alone. IRIS relies on GPT-4 via the cloud and targets Java; this work adopts a conceptually similar combined approach but with local models targeting PHP.

LLM-assisted code transformation. Stümpfle et al. [12] used LLMs for software variant migration into a Software Product Line, employing a self-refinement loop that feeds compilation test results back to the model iteratively. The feedback loop serves a similar role to PHPStan post-conversion validation in this pipeline. A fundamental difference is focus: their work targets cross-variant feature extraction; this work targets PHP version migration and security compliance.

Long-term consequences of automated transformation. Strobl et al. [15] analyzed three government-sector automated COBOL/PL/I-to-Java conversions each exceeding one million lines, finding that the absence of embedded validation mechanisms caused problems that only emerged after deployment. This finding motivates the inclusion of PHPStan and dual Semgrep scans (pre- and post-conversion) in this pipeline.

AST-based migration at industrial scale. Chen et al. [5] demonstrated that AST-based symbol replacement can migrate large-scale TypeScript codebases at Microsoft with negligible functional regression, validating the reliability of the Rector-based approach adopted here. Neither Cai et al. [16] nor Chen et al. address security or standards compliance.

Taken together, no prior work combines PHP version migration, locally running LLM security analysis, post-conversion static validation, and ISO/IEC 27001:2022 compliance reporting in a single open-source workflow.

III. SYSTEM ARCHITECTURE

A. Pipeline Overview

The pipeline consists of six Python modules orchestrated by `main.py`. Modules execute sequentially; all inter-module communication passes as in-memory Python objects. The modular design allows any component to be replaced or improved independently. Fig. 1 shows the pipeline architecture.

B. Module Descriptions

Scanner (`scanner.py`) runs Semgrep with the `p/php` and `p/owasp-top-ten` rulesets on the `input/` directory before conversion (pre-scan), then again on `output/` after conversion (post-scan). The difference $\Delta F = F_{\text{post}} - F_{\text{pre}}$

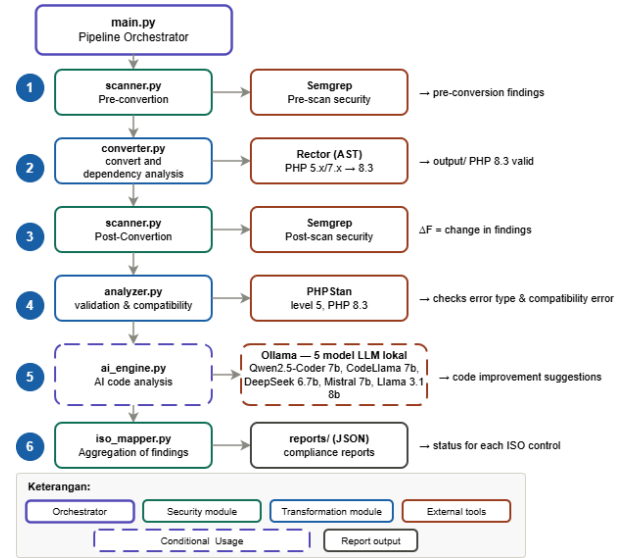


Fig. 1. Pipeline system architecture with six functional modules and one orchestrator (`main.py`).

measures whether the conversion introduced or eliminated vulnerabilities. Findings are classified by vulnerability type and mapped to ISO/IEC 27001:2022 controls via keyword matching on `rule_id` and `message` fields.

Converter (`converter.py`) copies all PHP files from `input/` to `output/`, then runs Rector with a runtime-generated configuration targeting the specified PHP version. Rector applies a cumulative `LevelSetList` chain from `UP_TO_PHP_53` through the target version plus `SetList::CODE_QUALITY`, `DEAD_CODE`, `EARLY_RETURN`, and `TYPE_DECLARATION` sets. The `input/` directory is never modified; failed files remain in `output/` as copies of the original.

Analyzer (`analyzer.py`) runs PHPStan level 5 on `output/` to verify type correctness and compatibility with the target PHP version. Each finding is tagged with the relevant ISO/IEC 27001:2022 control and classified as `ERROR` or `WARNING`.

AI Engine (`ai_engine.py`) receives Semgrep pre-scan findings with priority levels 1–5 (SQL Injection, XSS, deprecated functions, hardcoded credentials, path traversal / SSRF) and sends each one to the selected LLM via the Ollama REST API at `localhost:11434`. The prompt requests a vulnerability explanation, a secure PHP 8.x replacement in a fenced code block, and a `CONFIDENCE: [0–100]` line. Code blocks are extracted by markdown pattern matching and validated with `php -l`. The module is automatically skipped if no findings exceed the priority threshold or if Ollama is unavailable.

ISO Mapper (`iso_mapper.py`) collects all findings from the Scanner, Analyzer, and AI Engine as in-memory objects and applies deterministic rule-based mapping to six ISO/IEC 27001:2022 Annex A controls: A.5.17 (authentication information), A.8.24 (use of cryptography), A.8.25

(secure development lifecycle), A.8.26 (application security requirements), A.8.28 (secure coding), and A.8.29 (security testing in development). Rule-based mapping was chosen over AI-based mapping to ensure deterministic, auditable results. Per-control status is `COMPLIANT` (no findings), `NON_COMPLIANT` (active Semgrep findings), or `PARTIAL` (PHPStan-only findings).

Composer Analyzer (`composer_analyzer.py`) checks PHP 8.x compatibility of each dependency declared in `composer.json` using Packagist metadata. The module is automatically skipped if no `composer.json` is present.

`main.py` combines all module outputs into a structured JSON report in `reports/` and displays a Rich-formatted console summary. The JSON report is designed to serve directly as ISO/IEC 27001:2022 audit documentation. The complete pipeline source code is publicly available at <https://github.com/MFarrelAkbar1/skripsi-php-migration>.

C. LLM Evaluation Metrics

Five metrics are applied. The **Rector Conversion Rate** (CR) is $f_{\text{success}}/f_{\text{total}} \times 100\%$. The **Format Compliance Rate** (FCR) adapts prompt-level strict-accuracy from IFEval [17]: a response is compliant only if it contains all three required elements simultaneously (explanation, PHP code block, and `CONFIDENCE: line`). The **Syntax Validity Rate** (SVR) adapts `pass@1` [18]: the percentage of extracted code blocks that pass `php -l`. SVR is designated the primary metric because it is independently verifiable. The **Security Finding Delta** $\Delta F = F_{\text{post}} - F_{\text{pre}}$ measures the post-conversion security posture change. The **Mean Confidence Score** $\bar{C}_m$ is a secondary metric recording the model’s self-reported score; prior work confirms that small-to-medium LLMs are poorly calibrated [11] [10].

IV. RESULTS AND DISCUSSION

A. Datasets

Case Study 1 uses 245 PHP files from the TCK Dashboard application of FT UGM (Faculty of Engineering, Universitas Gadjah Mada), built on CodeIgniter 3.x with PHP 7.4.9. Case Study 2 uses 326 PHP files from the Computer-Based Test (CBT) application of DTI UGM (Directorate of Information Technology), built on CodeIgniter 3.x with a $\text{PHP} \geq 7.0$ requirement. Both codebases were used solely for static analysis in a local environment; no source code was transmitted to any external service.

B. Rector Conversion Results

Rector was run with a PHP 8.3 target on both datasets. Fig. 2 compares conversion results across both case studies.

FT UGM achieved a conversion rate of 97.6% (239 converted, 4 skipped, 2 failed). CBT DTI achieved 91.7% (299 converted, 25 skipped, 2 failed). The 5.9 percentage point difference is primarily attributable to a higher proportion of skipped files in CBT DTI (7.7% vs. 1.6%): skipped files already satisfy PHP 8.3 requirements and do not require transformation.

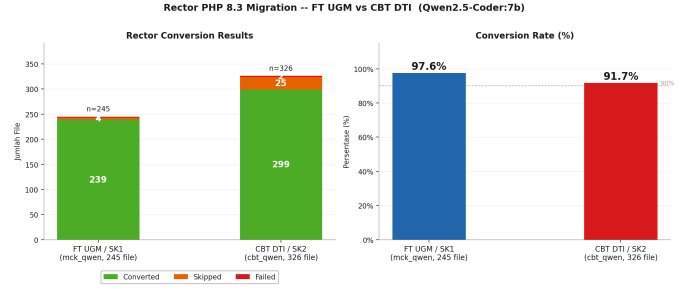


Fig. 2. Comparison of Rector conversion results for both case studies (FT UGM: 245 files; CBT DTI UGM: 326 files).

In both datasets, exactly 2 files failed with the same internal `FiberNodeScopeResolver` bug in the Rector version used, triggered by branch patterns returning heterogeneous types (`captcha_helper.php`) or by large numbers of `if (!function_exists(...))` constructs (`form_helper.php`). This failure is systemic to the Rector version, not a property of the source code. All files in `output/` passed `php -l` after conversion, yielding a post-conversion syntax validity rate of 100% for both case studies.

Sampling-based accuracy validation on 24 randomly selected files (10% of valid files, fixed seed for reproducibility) found that Rector output and an LLM-assisted manual reference achieved identical syntax validity (100%) and deprecated removal rates (100%), with a mean line-level similarity score of 0.9214 (11 of 12 application-layer files in the 0.90–1.00 range).

C. Semgrep Security Scan Results

Semgrep produced 11 findings in each dataset: 1 SQL Injection and 10 SSRF, all concentrated in `system/core/Output.php`, a core component of CodeIgniter 3.x shared by both applications. $\Delta F = 0$ in both case studies confirms that Rector’s AST-based transformation introduced no new vulnerabilities. The pre-existing findings require remediation at the framework code level, which lies outside the scope of syntactic transformation.

D. PHPStan Validation Results

PHPStan level 5 targeting PHP 8.3 produced 2,790 findings for FT UGM and 2,354 for CBT DTI. In both datasets, a substantial fraction are false positives arising from PHPStan’s inability to trace CodeIgniter 3.x’s dynamic architecture (magic properties such as `$this->db` and runtime-loaded helper functions such as `base_url()`): 354 and 318 items, respectively. The remaining errors reflect technical debt—type inconsistencies, undeclared return types, and missing dependency classes—that existed prior to migration but were masked by PHP 7.x’s permissive type system. The higher error count in FT UGM despite its smaller file count is explained by the use of third-party libraries (`PhpOffice\PhpSpreadsheet`, `Dotenv\Dotenv`) whose classes PHPStan cannot resolve without a full dependency installation. These findings are consistent with Strobl et al. [15], who observed that automated

transformation exposes latent code quality issues that only become apparent under stricter runtime environments.

E. ISO/IEC 27001:2022 Compliance Status

Table I summarizes the primary pipeline metrics and ISO compliance status for both case studies.

TABLE I
PIPELINE METRICS AND ISO/IEC 27001:2022 COMPLIANCE STATUS FOR BOTH CASE STUDIES

Metric	FT UGM	CBT DTI
PHP files analyzed	245	326
Rector CR	97.6%	91.7%
Post-conv. syntax validity	100%	100%
Semgrep findings (pre)	11	11
SQL Injection	1	1
SSRF	10	10
ΔF	0	0
PHPStan findings	2,790	2,354
A.5.17 Auth. Info.	COMPLIANT	COMPLIANT
A.8.24 Cryptography	COMPLIANT	COMPLIANT
A.8.25 Dev. Lifecycle	NON_COMPL.	NON_COMPL.
A.8.26 App. Security	NON_COMPL.	NON_COMPL.
A.8.28 Secure Coding	NON_COMPL.	NON_COMPL.
A.8.29 Security Testing	NON_COMPL.	NON_COMPL.
ISO overall status	NON_COMPL.	NON_COMPL.

The NON_COMPLIANT status accurately reflects the pre-remediation security posture of legacy codebases, not a pipeline failure. COMPLIANT status for A.5.17 and A.8.24 indicates that the Semgrep ruleset produced no hardcoded credential or weak cryptography findings in either dataset. The JSON reports generated automatically on each pipeline execution are structured for direct use as ISO/IEC 27001:2022 audit documentation, providing per-finding mappings to Annex A controls without additional manual work.

F. LLM Comparison Experiment

The comparison experiment used the 11 Semgrep pre-scan findings from FT UGM as input, with 3 runs per finding per model (33 trials per model, 165 total) at a temperature of 0.1. Condition A applied an identical zero-shot prompt to all models. Condition B applied a per-model optimized prompt based on failure patterns observed in Condition A.

Fig. 3 shows the Format Compliance Rate for Condition A. Fig. 4 shows the Syntax Validity Rate and code block extraction rate for Condition A. Table II summarizes FCR and SVR across both conditions for all five models.

The results reveal a systematic trade-off between FCR and SVR. CodeLlama achieved 100% FCR in both conditions but its SVR was only 27.3% (Condition A) and 39.4% (Condition B): the model follows format instructions consistently but generates syntactically invalid PHP code in the majority of cases. Qualitative inspection shows that CodeLlama correctly identifies vulnerabilities but produces code blocks that fail to close properly, or reformats code without actually fixing the issue.

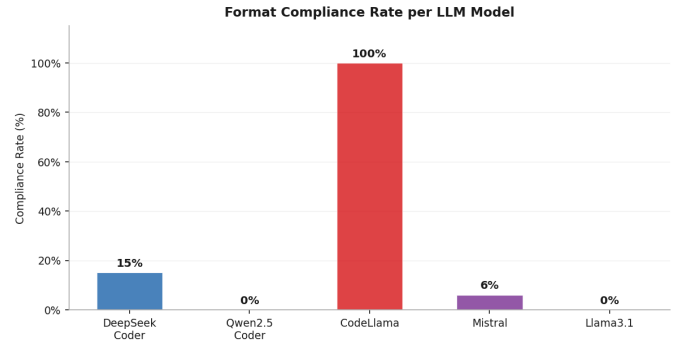


Fig. 3. Format Compliance Rate (FCR) for all five LLM models in Condition A (zero-shot, identical prompt, $n = 33$ per model).

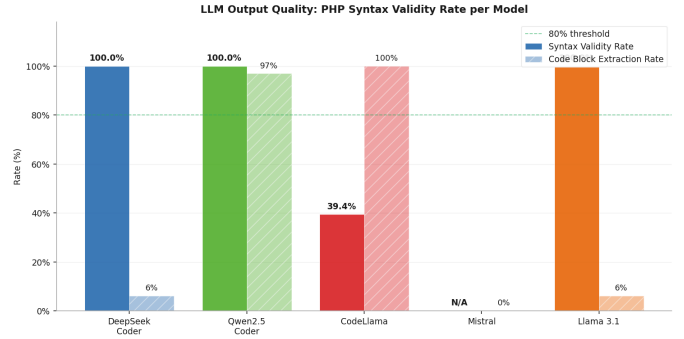


Fig. 4. Syntax Validity Rate (SVR) and code block extraction rate for all five LLM models in Condition A.

Qwen2.5-Coder achieved 0% FCR (it ignores format instructions entirely) yet produced 97–100% extraction rates and 100% SVR in both conditions. Its remediation suggestions also demonstrate deeper semantic understanding: for the SQL Injection finding in `Output.php` at line 768, Qwen2.5-Coder recognized that the flagged code constructs a cache path rather than an SQL query, and correctly applied `urlencode()` to the `$_SERVER['QUERY_STRING']` input.

Prompt optimization in Condition B produced significant improvement only for models with sufficient baseline instruction-following capability. DeepSeek Coder improved from 15% to 61% FCR when given shorter, more direct instructions, confirming it was burdened by prompt complexity

TABLE II
FCR AND SVR COMPARISON: CONDITION A VS. CONDITION B FOR ALL FIVE LLM MODELS ($n = 33$ PER MODEL PER CONDITION)

Model	FCR		SVR	
	Cond. A	Cond. B	Cond. A	Cond. B
DeepSeek Coder 6.7b	15%	61%	100%	100%
Qwen2.5-Coder 7b	0%	0%	100%	100%
CodeLlama 7b	100%	100%	27.3%	39.4%
Mistral 7b	6%	0%	0%	0%
Llama 3.1 8b	0%	3%	100%	0%

SVR measured on ≤ 3 extracted blocks; not statistically representative.

rather than lacking capability. The few-shot strategy applied to Mistral had the opposite effect (FCR dropped from 6% to 0%), consistent with findings in the literature that prompt engineering techniques effective for large models cannot always be transferred to small-to-medium models [11]. CodeLlama’s SVR limitation proved structural: its PHP 8.x code generation quality did not materially improve despite explicit syntax validation instructions.

Parameter sensitivity experiments (Condition C) on Qwen2.5-Coder confirmed that its SVR is stable at 100% across temperatures 0.1–0.7. Increasing the context window from 2,000 to 4,000 characters raised the extraction rate from 97% to 100%, suggesting a marginal benefit for longer source files.

The mean confidence score $\bar{C}_m$ showed no reliable correlation with code quality in any condition. CodeLlama reported $\bar{C}_m = 0.909$ while producing primarily invalid code; Qwen2.5-Coder reported $\bar{C}_m = 0.000$ (no parseable confidence line) while achieving 100% SVR. This confirms that SVR—not FCR or $\bar{C}_m$ —is the most valid evaluation metric for locally running LLM-based PHP security analysis systems, consistent with calibration findings in [11] and [19].

V. CONCLUSIONS

This paper presented a Python-based pipeline that unifies automated PHP legacy code migration, security scanning, and ISO/IEC 27001:2022 compliance reporting into a single open-source, fully local workflow. Four main conclusions follow.

First, Rector-based AST transformation achieved conversion rates of 97.6% and 91.7% across two real-world CodeIgniter 3.x codebases with 100% post-conversion syntax validity in both cases. The two failures per dataset were caused by a reproducible Rector bug, not source code characteristics, confirming the reliability of the approach for institutional codebases.

Second, $\Delta F = 0$ in both case studies demonstrates that automated AST-based transformation does not introduce new security vulnerabilities—an important guarantee for organizations whose operational continuity depends on the integrity of migrated code. Automatically generated JSON reports with per-finding ISO/IEC 27001:2022 control mappings serve directly as audit documentation, eliminating the separate manual compliance step currently required by organizations pursuing certification.

Third, PHPStan validation exposed thousands of type compatibility issues that PHP 7.x’s permissive type system had concealed, providing concrete pre-deployment evidence of technical debt consistent with the findings of Strobl et al. [15].

Fourth, among five locally running open-source LLMs evaluated under zero-shot and per-model optimized conditions, Qwen2.5-Coder 7b is recommended for pipeline integration: it achieved SVR = 100% in every experimental condition despite ignoring format instructions, and demonstrated superior semantic understanding in remediation suggestions. FCR and SVR proved to be independent, uncorrelated dimensions; SVR

is the more practically meaningful metric for automated code quality assessment.

Future work should strengthen LLM evaluation with automated semantic test suites, develop CodeIgniter 3.x stubs to reduce PHPStan false positives, and extend the pipeline to support CI/CD integration and additional programming languages.

ACKNOWLEDGMENT

The authors thank FT UGM and DTI UGM for providing access to their PHP application source code for static analysis. All analysis was conducted locally; no source code was transmitted to external services.

REFERENCES

- [1] W3Techs, “Usage statistics of php,” 2024. [Online]. Available: <https://w3techs.com/technologies/details/pl-php>
- [2] The PHP Group, “PHP: Supported versions,” php.net, 2025, [Online]. Available: <https://www.php.net/supported-versions.php>. [Accessed: Apr. 2026].
- [3] National Institute of Standards and Technology, “National vulnerability database (NVD),” NIST, 2024, [Online]. Available: <https://nvd.nist.gov/>. [Accessed: Apr. 2026].
- [4] Rector, “Rector — instant upgrades and automated refactoring,” getreector.com, 2024, [Online]. Available: <https://getreector.com>. [Accessed: Apr. 2026].
- [5] D. Chen, D. Lawler, Y. Zhang, P. Kuchhal, D. Westcott, and G. Yang, “AST-based source code migration through symbols replacement,” in *Proc. 2022 IEEE Asia-Pacific Conf. on Computer Science and Data Engineering (CSDE)*, Brisbane, Australia, 2022, pp. 1–6.
- [6] Semgrep, “Semgrep — lightweight static analysis for many languages,” semgrep.dev, 2024, [Online]. Available: <https://semgrep.dev>. [Accessed: Apr. 2026].
- [7] PHPStan, “PHPStan — php static analysis tool,” phpstan.org, 2024, [Online]. Available: <https://phpstan.org>. [Accessed: Apr. 2026].
- [8] International Organization for Standardization and International Electrotechnical Commission, “ISO/IEC 27001:2022 information security, cybersecurity and privacy protection — information security management systems: Requirements, 3rd ed.” ISO/IEC, Geneva, 2022.
- [9] Ollama, “Ollama — get up and running with large language models locally,” ollama.com, 2024, [Online]. Available: <https://ollama.com>. [Accessed: Apr. 2026].
- [10] Z. Li, S. Dutta, and M. Naik, “IRIS: LLM-assisted static analysis for detecting security vulnerabilities,” in *Proc. 13th Int. Conf. on Learning Representations (ICLR)*, Singapore, 2025, [Online]. Available: <https://arxiv.org/abs/2405.17238>.
- [11] V. Akuthota, R. Kasula, S. T. Sumona, M. Mohiuddin, M. T. Reza, and M. M. Rahman, “Vulnerability detection and monitoring using LLM,” in *Proc. 2023 IEEE 9th Int. Women in Engineering (WIE) Conf. on Electrical and Computer Engineering (WIECON-ECE)*, Thiruvananthapuram, India, 2023, pp. 309–314.
- [12] J. Stümpfle, D. Atray, N. Jazdi, and M. Weyrich, “Large language model assisted transformation of software variants into a software product line,” in *Proc. 2025 IEEE/ACM 22nd Int. Conf. on Software and Systems Reuse (ICSR)*, Ottawa, Canada, 2025, pp. 12–20.
- [13] E. Merlo, D. Letarte, and G. Antoniol, “Automated protection of PHP applications against SQL-injection attacks,” in *Proc. 11th European Conf. on Software Maintenance and Reengineering (CSMR)*, Amsterdam, Netherlands, 2007, pp. 191–202.
- [14] C. Li, B. Guan, Q. Zheng, B. Liu, and Z. Zhang, “Software vulnerability detection based on LLM,” in *Proc. 2025 IEEE 7th Advanced Information Management, Communications, Electronic and Automation Control Conf. (IMCEC)*, Chongqing, China, 2025, pp. 1534–1539.
- [15] S. Strobl, C. Zoffi, C. Haselmann, M. Bernhart, and T. Grechenig, “Automated code transformations: Dealing with the aftermath,” in *Proc. 2020 IEEE 27th Int. Conf. on Software Analysis, Evolution and Reengineering (SANER)*, London, Canada, 2020, pp. 627–631.

- [16] Z. Cai, L. Zhao, X. Wang, X. Yang, J. Qin, and K. Yin, "A pattern-based code transformation approach for cloud application migration," in *Proc. 2015 IEEE 8th Int. Conf. on Cloud Computing (CLOUD)*, New York, USA, 2015, pp. 33–40.
- [17] J. Zhou, T. Lu, S. Mishra, S. Brahma, S. Basu, Y. Luan, D. Zhou, and L. Hou, "Instruction-following eval for large language models," arXiv preprint arXiv:2311.07911, 2023, [Online]. Available: <https://arxiv.org/abs/2311.07911>.
- [18] B. Rozière *et al.*, "Code Llama: Open foundation models for code," *arXiv preprint arXiv:2308.12950*, 2024, [Online]. Available: <https://arxiv.org/abs/2308.12950>.
- [19] K. Yan, H. Guo, X. Shi, S. Cao, D. Di, and Z. Li, "CodeIF: Benchmarking the instruction-following capabilities of large language models for code generation," arXiv preprint arXiv:2502.19166, 2025, [Online]. Available: <https://arxiv.org/abs/2502.19166>.